AROR UNIVERSITY

LEARN • INNOVATE • LEAD

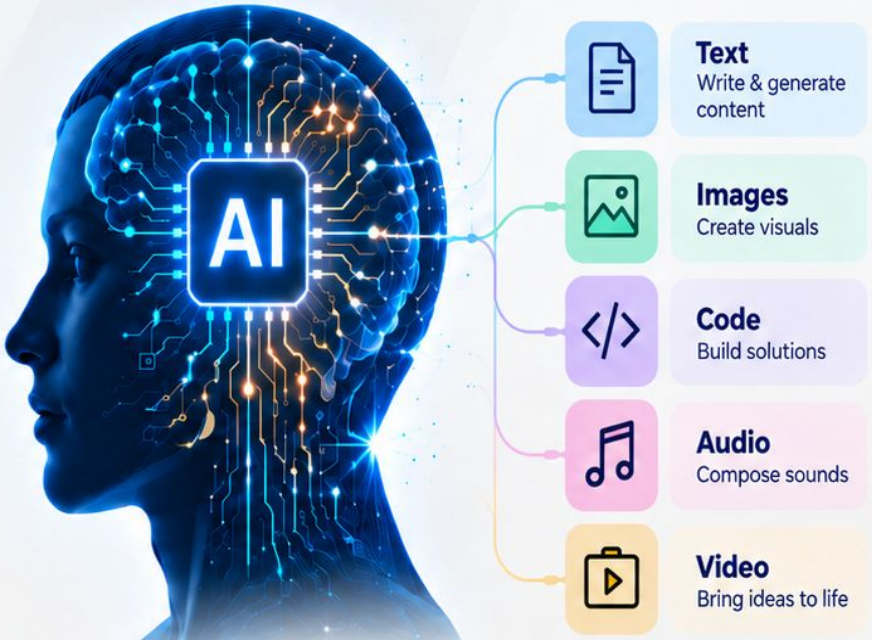
"Building Intelligent Tomorrows"

ARI-406 | GENERATIVE AI COURSE

LECTURES 7–9

Generative AI

From Text Representation to Language Modelling



From Text to Machine Representation
Text preprocessing, vocabulary, tokens & token IDs, embeddings

Tokenization
Word-level & character-level tokenization, subword tokenization, Byte Pair Encoding, WordPiece/SentencePiece, vocabulary-size trade-offs

Language Modelling
Language models, context, next-token prediction, probability over sequences, context windows, generation process

COURSE INSTRUCTOR

Dr. Rajesh Kumar

Assistant Professor
University of New York Tirana

Adjunct Faculty Member
AROR University

Create • Innovate • Transform

Generative AI

- From Text to Machine Representation


TEXT PREPROCESSING
The cat's running, quickly!
It chased 2 mice across the garden.
...
the cat is running quickly
it chased two mice across the garden
VOCABULARY
the ...
cat ...
is ...
running ...
quickly ...
it ...
chased ...
two ...
mice ...
across ...
garden ...
TOKENS & TOKEN IDs
the → 101
cat → 4932
is → 17
running → 8721
quickly → 6203
it → 33
chased → 9104
two → 428
mice → 7771
across → 306
garden → 5190
EMBEDDINGS
101
4932
17
8721
6203
33

Lecture Objectives

- By the end of this lecture, you should be able to:
- Explain why text must be converted into machine-readable representations
- Describe basic text preprocessing
- Define a vocabulary
- Explain tokens and token IDs
- Explain the purpose of embeddings
- Trace the path from text to vector representation

How Can a Machine Understand Text?

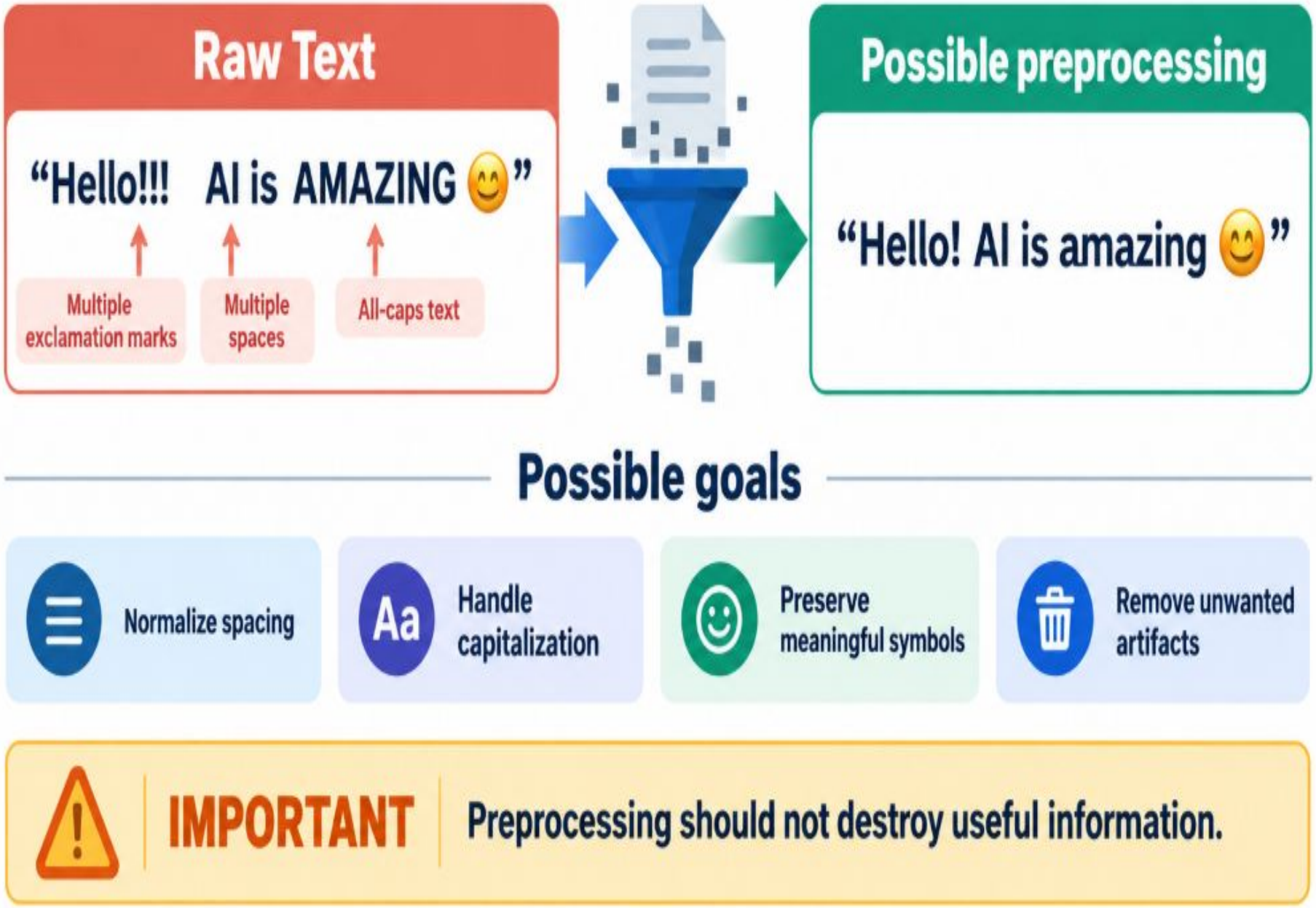
- Human:
- “Generative AI is powerful.”
- Machine needs:
- Numbers
- **Basic Pipeline**
- **Raw Text**
  ↓
  Preprocessing
  ↓
  Tokens
  ↓
  Token IDs
  ↓
  Embeddings
  ↓
  Neural Network

Why Text Representation Matters

- Language contains:
- Words and phrases
- Grammar and structure
- Context
- Relationships
- Meaning
- A useful representation should help a model capture:
- **Structure + Similarity + Context + Meaning**

Text Preprocessing

- Before text is used by a model, we may need to:
- Clean unwanted text
- Normalize formatting
- Handle whitespace
- Handle punctuation
- Standardize selected patterns
- Remove or transform irrelevant content
- **Goal**
- Convert messy text into a more consistent form


Raw Text
“Hello!!! AI is AMAZING 😊”
Multiple exclamation marks
Multiple spaces
All-caps text
Possible preprocessing
“Hello! AI is amazing 😊”
Possible goals
Normalize spacing
Aa
Handle capitalization
Preserve meaningful symbols
Remove unwanted artifacts
IMPORTANT
Preprocessing should not destroy useful information.

Text and the Idea of a Vocabulary

What Is a Vocabulary?

A vocabulary is a collection of discrete text units that a model can represent.



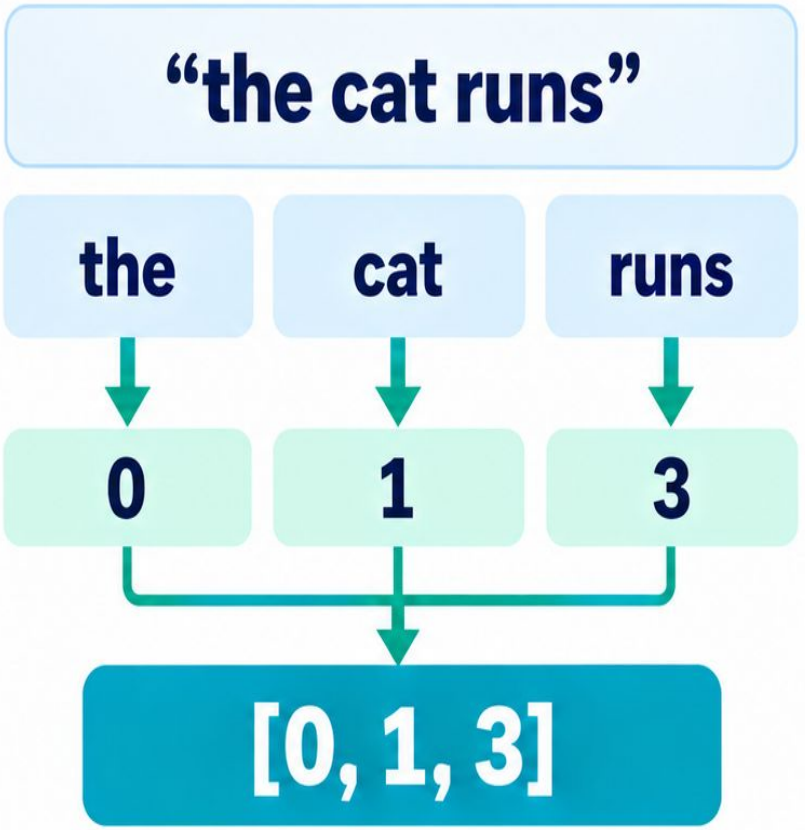
The vocabulary defines the available units.

Vocabulary as a Mapping

Vocabulary = Mapping

Token	Token ID
the	0
cat	1
dog	2
runs	3
AI	4

Example



What Is a Token?

- **Token**
- A **token** is a discrete unit of text used by the model.
- Depending on the tokenizer, a token can represent:
- A word
- Part of a word
- A character
- Punctuation
- A special symbol
- Example:
- “AI is powerful.”
- Possible tokens:
- AI | is | powerful | .

Activity: Word or Token?

- Consider:
- **“unhappiness”**
- Could it be represented as:
- **A.** unhappiness
- **B.** un + happiness
- **C.** several smaller pieces
- **Question:**
- **Does one word always equal one token?**

What Is Tokenization?

- **Tokenization = dividing text into discrete units called tokens**
- Example:
- **Generative AI is powerful.**
- Possible tokens:
- Generative | AI | is | powerful | .
- **Key Idea**
- **Text → Tokens**

Activity:

- How Many Tokens?
- Consider:
- **I love Generative AI!**
- Imagine three possibilities:
- **A. Word-level**
- I | love | Generative | AI | !
- **B. Character-level**
- I | | l | o | v | e | ...
- **C. Subword-level**
- I | love | Gener | ative | AI | !
- **Question**
- **Which representation would you choose? Why?**



Activity – How Many Tokens?

Consider: “I love Generative AI!”

A. Word-level

I	love	Generative	AI	!

B. Character-level

I	␣	l	o	v	e	…

C. Subword-level

I	love	Gener	ative	AI	!

? Question Which representation would you choose? Why?

Method	Vocabulary	Sequence Length	Rare Words
Word-level	Large	Short	Difficult
Character-level	Small	Long	Easy
Subword-level	Medium	Medium	Good

Key Idea
Subword tokenization provides a practical balance between vocabulary size, sequence length, and word coverage.

- **Tokens vs. Words**
- **Token ≠ Word**
- **Word-level representation**
- “unhappiness”
- → one word
- **Subword representation**
- “un” + “happiness”
- or another tokenizer-specific split
- **Why?**
- Smaller vocabulary
- Handles rare words
- Can represent new words
- Reuses common pieces

Tokens vs. Words

Word-level representation

“unhappiness”

one word

≠

Subword representation

“un” + “happiness”

or another tokenizer-specific split

Why?



Smaller vocabulary



Handles rare words



Can represent new words



Reuses common pieces

From Tokens to Token IDs

Tokens are converted into integers.

Example vocabulary

Token	Token ID
I	0
like	1
AI	2
today	3

Sentence

"I like AI"

Token sequence

I → 0

like → 1

AI → 2

[I, like, AI]

Token IDs

[0, 1, 2]

Why Token IDs Are Not Enough

- **Are Token IDs Meaningful?**
- Suppose:
- cat → 10
- dog → 11
- airplane → 12
- Does this mean:
- **dog is closer to cat than airplane?**
- No.
- Token IDs are:
- Arbitrary
- Discrete
- Identifiers only
- We need a richer representation.

The Unknown Word Problem

- **The Vocabulary Problem**
- Suppose the vocabulary contains:
- play
  played
  playing
- Now the model encounters:
- **playfulness**
- What happens if the complete word is not in the vocabulary?
- **Possible result:**
- **<UNK>**
- **Problem**
- A new or rare word may become an **unknown token**.

Activity: Word-Level Challenge

- Suppose we want to represent:
- **connect, connected, connecting, connection, connectivity**
- **Question 1**
- With word-level tokenization, how many vocabulary entries might we need?
- **Question 2**
- What information is being repeated?
- **Question 3**
- Could smaller reusable pieces help?

Answer — Vocabulary Challenge

- **Q1. How many vocabulary entries might we need?**
  5 entries — one for each complete word.
- **Q2. What information is being repeated?**
  The common part **“connect”** appears repeatedly.
- **Q3. Could smaller reusable pieces help?**
  Yes. We could reuse pieces such as:
- connect | ed | ing | ion | ivity
- **Key Idea**
- **Word-level:** 5 separate entries
  Subword: Reuse common pieces → **smaller, more efficient vocabulary**

Answer – Vocabulary Challenge

Q1. How many vocabulary entries might we need?

5 entries

Q2. What information is being repeated?

The common part "connect" appears repeatedly.

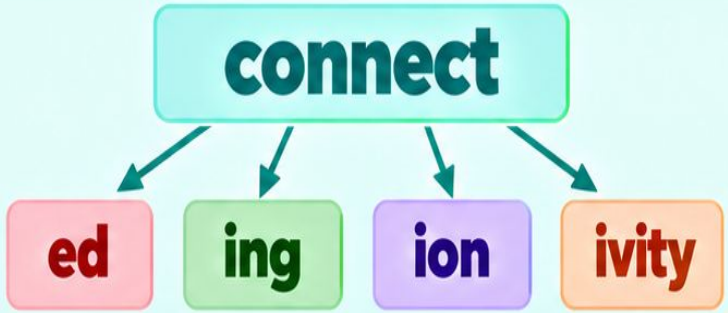
Q3. Could smaller reusable pieces help?

Yes.

Word-level: 5 separate entries

- connect
- connected
- connecting
- connection
- connectivity

Subword: Reusable pieces



connect | ed | ing | ion | ivity

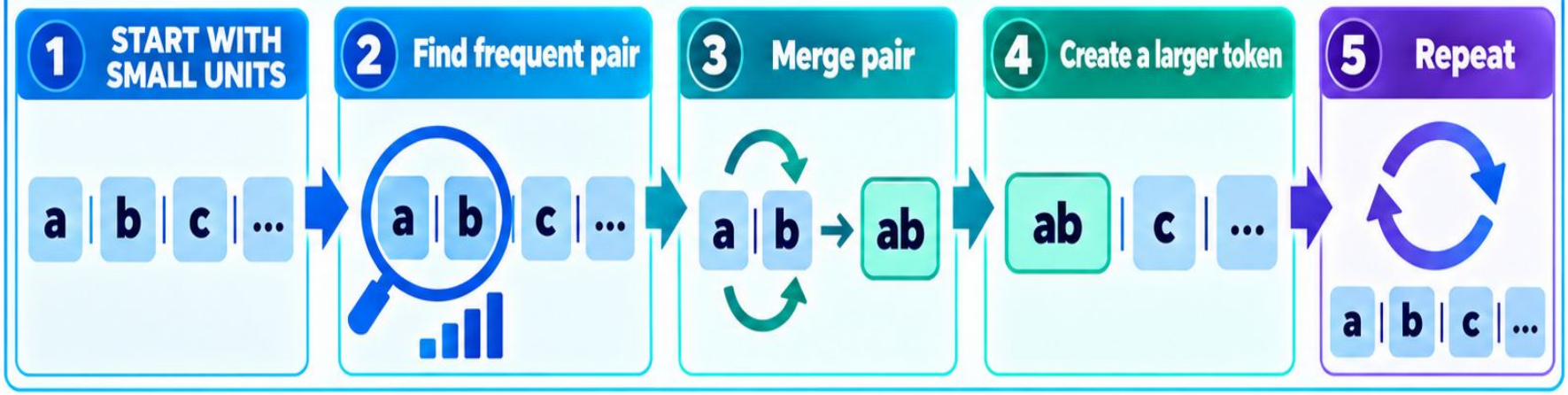
Key idea: Reuse common pieces → smaller, more efficient vocabulary

Byte Pair Encoding (BPE)

Basic Idea

Start with small units.
Repeatedly identify frequent sequences and merge them.

Conceptual Process



Result



A vocabulary containing useful subword units

WordPiece

WordPiece

WordPiece is another subword tokenization approach.

Main Idea

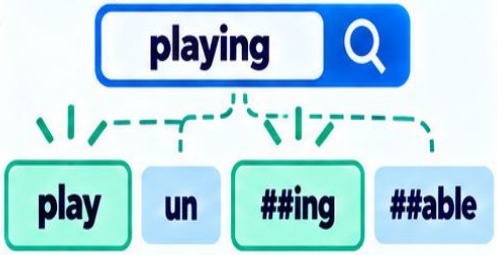
1 Build a Vocabulary

Build a vocabulary of useful subword units.



2 Select Useful Pieces

The tokenizer selects pieces that provide an effective representation of text.



3 Represent Text



Used in:

- Transformer-based language models
- Large-scale NLP systems

Key Idea

Represent words using reusable subword pieces

Character-Level Tokenization

Character-Level Tokenization

- Each character becomes a token.
- Example:
- **cat**
- c | a | t
- **Advantages**
- Very small vocabulary
- Rare words can still be represented
- No need for a huge word vocabulary
- **Limitations**
- Very long sequences
- Less direct linguistic meaning per token
- More computation may be required

Word-Level vs. Character-Level

Feature	Word-Level	Character-Level
Vocabulary size	Large	Small
Sequence length	Shorter	Longer
Rare words	Difficult	Easy to represent
Unknown words	Possible	Much less of a problem
Linguistic unit	Word	Character

Activity: Choose the Strategy

- **Activity — Which Tokenization?**
- Choose the most suitable strategy for each situation:
- **A**
- A very large dictionary-based NLP system
- **B**
- Text containing many rare words and names
- **C**
- A modern language model needing an efficient compromise
- **Choose:**
- **Word-Level**
  Character-Level
  Subword

- **Answer — Which Tokenization?**
- **A. Very large dictionary-based NLP system**
  ✅ Word-Level
- **B. Text containing many rare words and names**
  ✅ Character-Level
- **C. Modern language model needing an efficient compromise**
  ✅ Subword
- **Key Idea**
- **Word-Level → shorter sequences, larger vocabulary**
  Character-Level → smaller vocabulary, longer sequences
  Subword → practical balance between both

Lec#9

Language Modelling

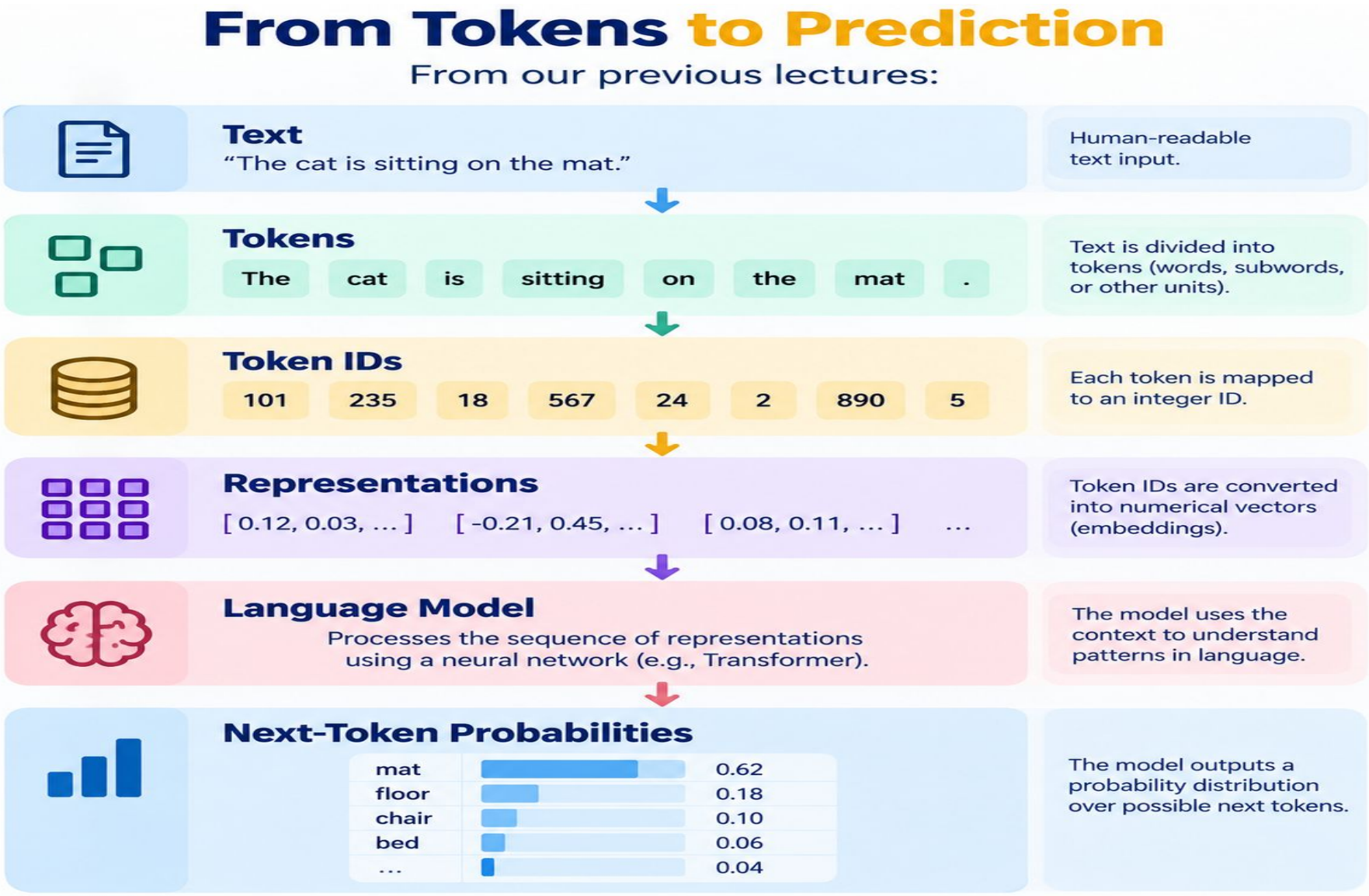
- How AI Learns to Predict and Generate Text

- **Learning Objectives**
- By the end of this lecture, you should be able to:
- Define a language model
- Explain the role of context
- Describe next-token prediction
- Interpret probabilities assigned to possible next tokens
- Explain probability over a sequence
- Define a context window
- Describe the basic text generation process

What Is a Language Model?

- A **language model** learns patterns in sequences of language.
- Given some text:
- **“The cat is sitting on the ___”**
- A language model estimates likely continuations:
- mat
- floor
- chair
- table
- bed
- **Core Question**
- **What is likely to come next?**

From Tokens to Language Modelling



Activity : Predict the Next Word

- **ou Are the Language Model**
- Complete this sentence:
- **“The sun rises in the _____.”**
- What would you predict?
- **A. east**
  B. kitchen
  C. computer
  D. sandwich
- Why did you choose that word?

Answer

- “The sun rises in the east.”
- Most likely choice:
- A. east
- Why?
- The previous words provide context.
- Key Idea
- Context influences what comes next.

What Is Context?

- **Context**
- **Context** is the information available to the model when making a prediction.
- Example:
- **“I went to the bank to deposit my _____.”**
- Likely:
- **money**
- But:
- **“We sat on the river bank and watched the _____.”**
- Likely:
- **water**
- **Same word — Different context**

Context Changes Prediction

- Consider:
- **“She poured coffee into the…”**
- Likely:
- **cup**
- Now:
- **“She poured water into the…”**
- Possible:
- **glass**
- **Prediction depends on:**
- Previous tokens
- Sentence structure
- Learned language patterns
- Available context

Next-Token Prediction

Given:
“Artificial intelligence is”
The model might predict:

Next Token	Probability
powerful	0.30
transforming	0.25
useful	0.20
changing	0.15
dangerous	0.10

The model produces a probability distribution.

Activity — Context Challenge

- Predict the next word:
- Sentence A
- “The student opened the book and began to _____.”
- Sentence B
- “The student opened the refrigerator and began to _____.”
- Question
- Why are your predictions different?

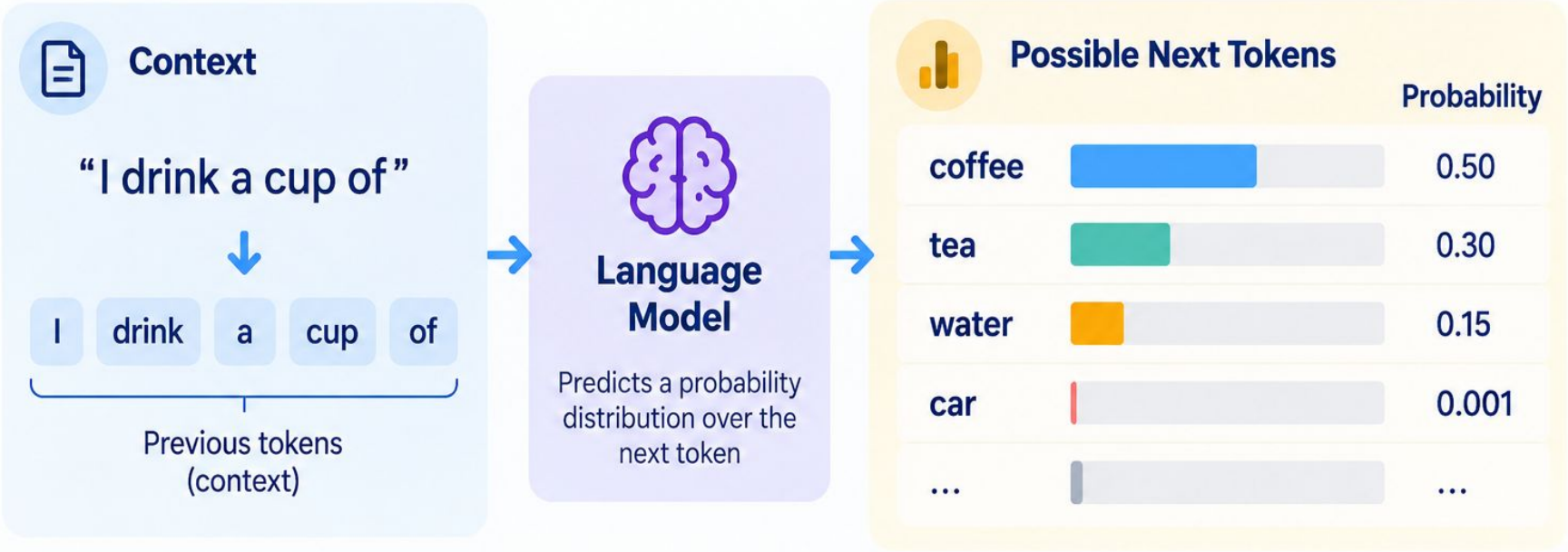
Context Challenge — Answer

- Possible predictions:
- **A**
- “The student opened the book and began to **read**.”
- **B**
- “The student opened the refrigerator and began to **look**.”
- **Why?**
- **Different context → Different next-token probabilities**
- **Key Idea**
- The model predicts the next token **conditioned on the preceding tokens**.

Next-Token Probability

The model estimates the probability of each possible next token, given the previous tokens.

P(next token | previous tokens)



Key Idea: The probability of the next token depends on the previous tokens (context).

Does Highest Probability Always Win?

Not necessarily! We can either choose the most likely token, or sample from the distribution.

Context

"I drink a cup of"

Model output: P(next token | context)

Token	Probability	Visualisation
coffee	0.50	
tea	0.30	
water	0.15	
juice	0.05	

Choose the highest
(Greedy decoding)

Always pick the token with the highest probability.

Selected token:

coffee

(P = 0.50)

Sample from the distribution

Randomly choose a token based on the probabilities.

Higher-probability tokens are more likely, but lower-probability tokens can still be selected.

Possible selected tokens:

- coffee (P = 0.50)
- tea (P = 0.30)
- water (P = 0.15)
- juice (P = 0.05)

Key Idea: The highest-probability token is not the only option. Sampling allows for more varied and creative outputs.

Probability Over Sequences

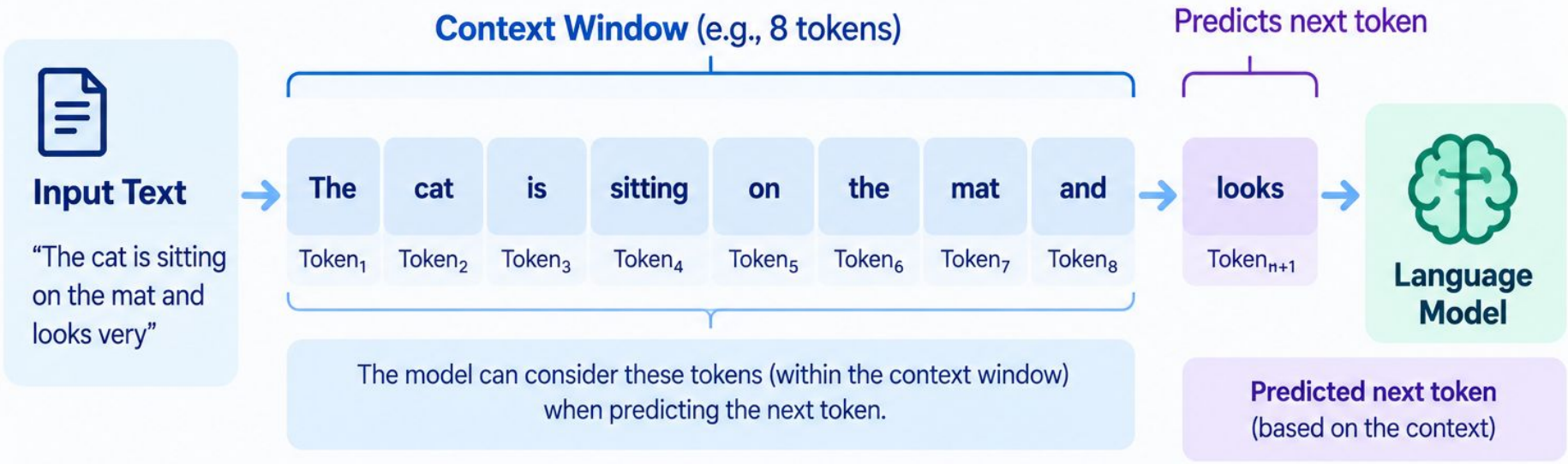
- Language models do not only consider isolated words.
- Consider:
- **“AI is changing the world”**
- A sequence can be viewed as a series of predictions:
- **P(AI)**
- ×
- **P(is | AI)**
- ×
- **P(changing | AI is)**
- ×
- **P(the | AI is changing)**
- ×
- **P(world | AI is changing the)**

- **Activity 3: Which Sequence Is More Likely?**
- 🎯 **Activity — Compare the Sequences**
- Which sequence would likely receive a higher probability?
- **A**
- **“The cat drank the milk.”**
- **B**
- **“The cat drank the laptop.”**
- **Question**
- Why?

- **Activity 3: Answer**
- ✅ **Answer**
- More likely:
- **“The cat drank the milk.”**
- Why?
- It better matches learned language patterns and context.
- **Key Idea**
- A language model assigns probabilities to **possible sequences**.
- **Higher probability ≠ guaranteed truth**

What Is a Context Window?

The **context window** defines how much token information the model can consider at one time.



Context window

Defines the maximum number of tokens the model can consider at one time.

Measured in tokens

Context windows are measured in tokens, not simply words. A single word can be multiple tokens.

Key Idea

The model uses the available context (previous tokens) to predict the next token.

Why Context Window Matters

- A larger usable context can provide:
- More conversation history
- Longer documents
- More instructions
- More examples
- Additional relevant information
- But longer context also creates challenges:
- More computation
- More memory
- Relevant information may be harder to use effectively

Activity Context Window Problem

- 🧩 **Activity — What Would You Keep?**
- Imagine the model has limited context space.
- You are building a:
- **University Information Assistant**
- The student asks:
- **“What are the prerequisites for ARI-406?”**
- Which context is most useful?
- **A. University sports history**
- **B. ARI-406 course information**
- **C. Cafeteria menu**
- **D. Student society announcements**
- **Why?**

Activity : Answer

- ✅ **Answer**
- **Best Context:**
- **B. ARI-406 course information**
- Why?
- It is directly relevant to the student's question.
- **Key Principle**
- **Useful context is not just MORE context.**
- It is:
- **Relevant Context**

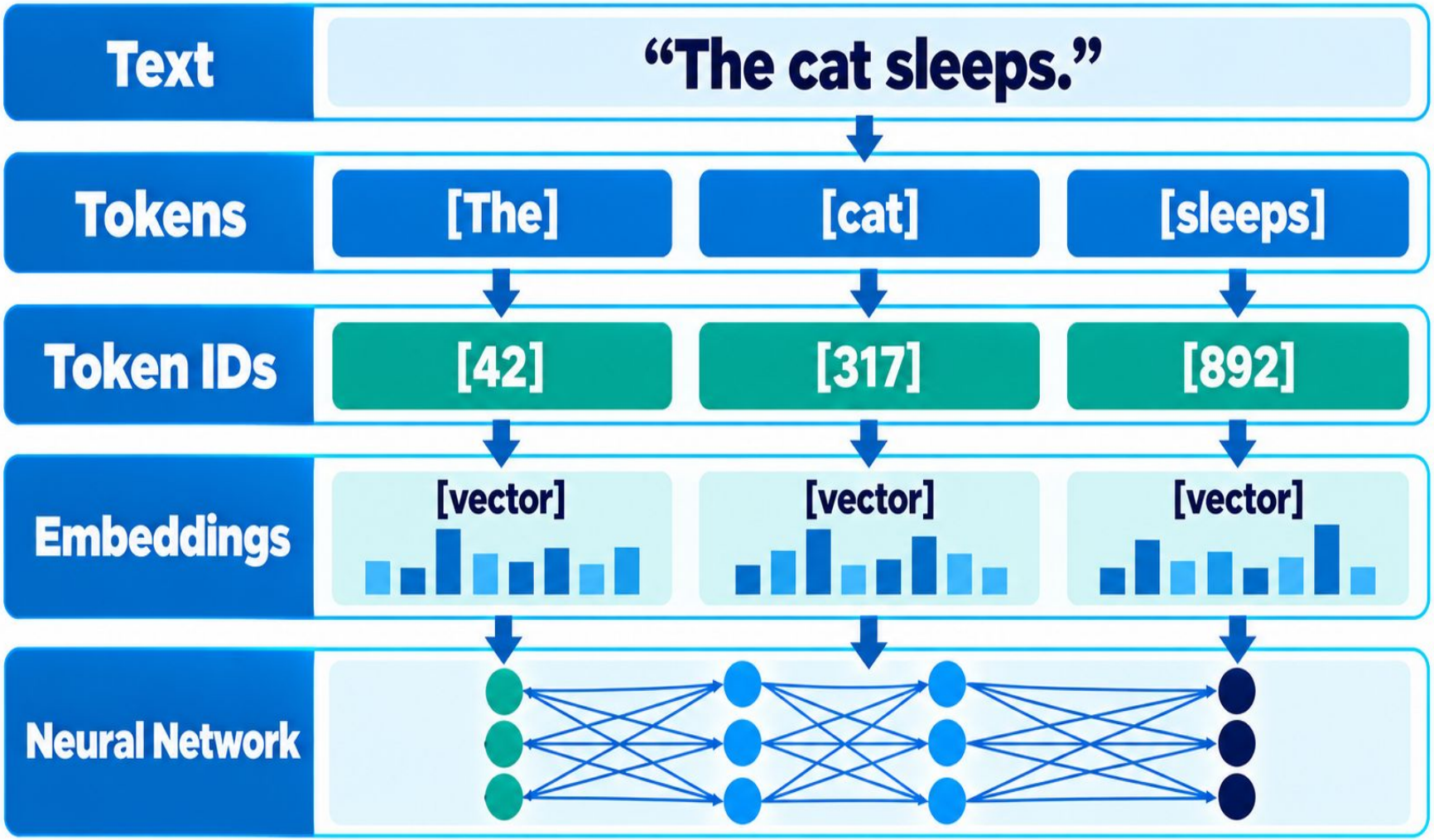
Research Paper

What Is an Embedding?

- **Embeddings**
- An **embedding** is a dense numerical vector representing a token or other linguistic unit.
- Example:
- **cat**
- → [0.21, -0.47, 0.83, ...]
- **dog**
- → [0.19, -0.44, 0.79, ...]
- **airplane**
- → [-0.62, 0.31, -0.18, ...]
- Embeddings provide a richer representation than token IDs.

From ID to Embedding

The Representation Pipeline



A Simple End-to-End Example

Sentence

“Generative AI creates content.”

Step 1 — Text

Generative AI creates content.

Step 2 — Tokens

Generative → AI → creates → content → .

Step 3 — Token IDs

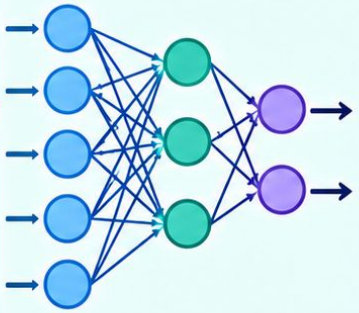
$[ID_1]$ → $[ID_2]$ → $[ID_3]$ → $[ID_4]$ → $[ID_5]$

Step 4 — Embeddings

$[V_1]$ → $[V_2]$ → $[V_3]$ → $[V_4]$ → $[V_5]$

Step 5

Embeddings → Neural Network



A Simple End-to-End Example

Sentence

"Generative AI creates content."

Step 1 — Text

Generative AI creates content.

Step 2 — Tokens

Generative

AI

creates

content

.

Step 3 — Token IDs

$[ID_1]$

$[ID_2]$

$[ID_3]$

$[ID_4]$

$[ID_5]$

Step 4 — Embeddings

$[V_1]$ 

$[V_2]$ 

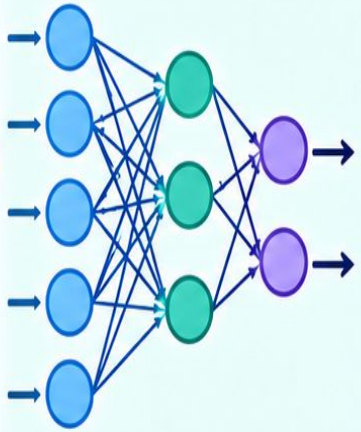
$[V_3]$ 

$[V_4]$ 

$[V_5]$ 

Step 5

Embeddings → Neural Network



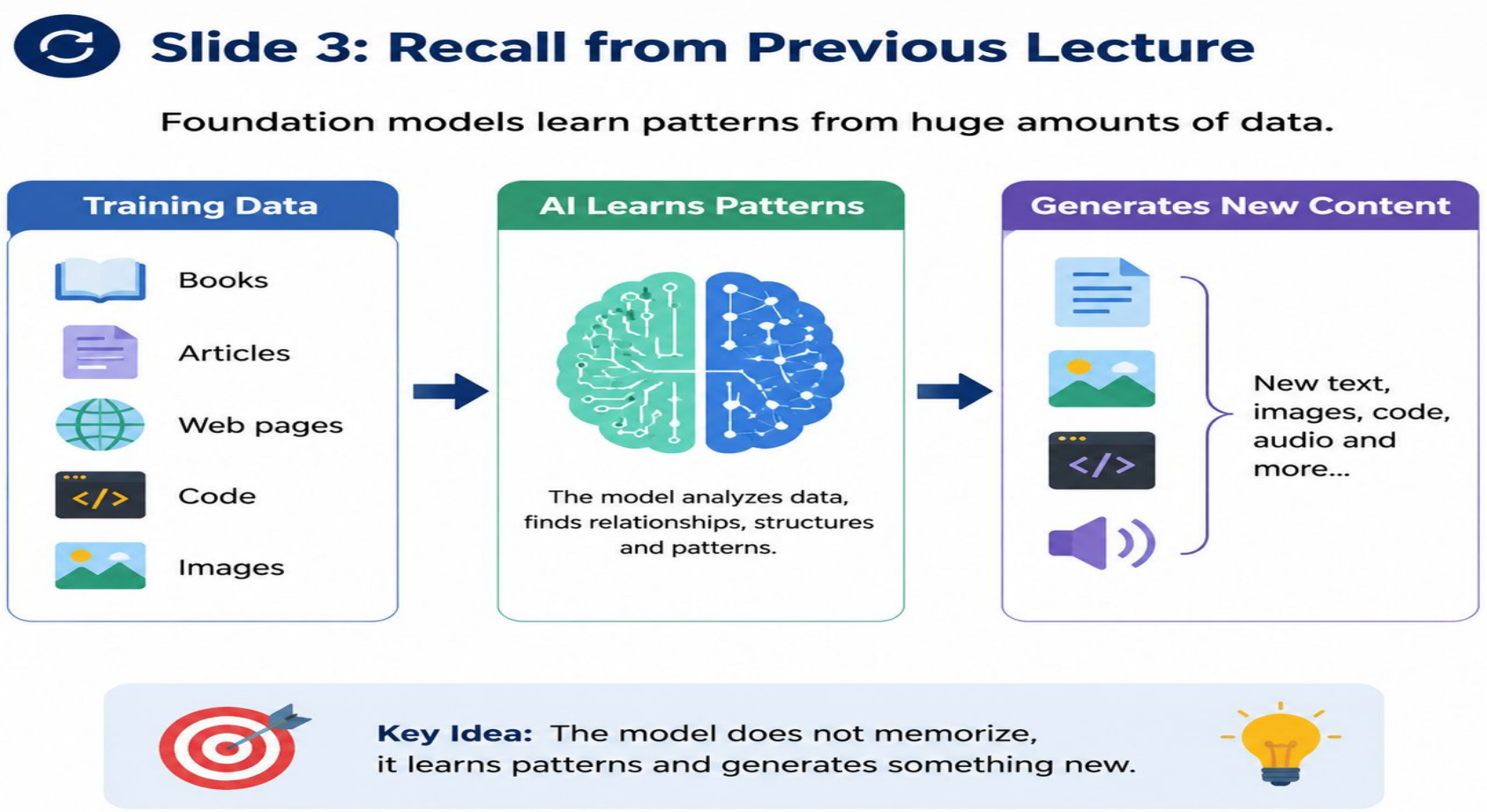
Connection to Large Language Models

- Modern language models require:
- **Text**
- ↓
- **Token Representation**
- ↓
- **Numerical Vectors**
- ↓
- **Transformer**
- ↓
- **Predictions**
- ↓
- **Generated Text**
- **Today's concepts form the foundation for:**
- Tokenization
- Language Modelling
- Transformers
- LLMs

Check Your Understanding

- **Q1.** Why does text need numerical representation?
- **Q2.** What is a vocabulary?
- **Q3.** Is every word necessarily one token?
- **Q4.** What is a token ID?
- **Q5.** Why are token IDs not sufficient?
- **Q6.** What does an embedding represent

Why Probability in Generative AI?



How does a model decide what to generate?

- If ChatGPT generates:
- "Artificial intelligence is changing the world..."
- How does the model decide which word comes next?
- Possible next words:
- rapidly
- today
- because
- through

The model does not "know".

It calculates probabilities.

Slide 5: Key Idea

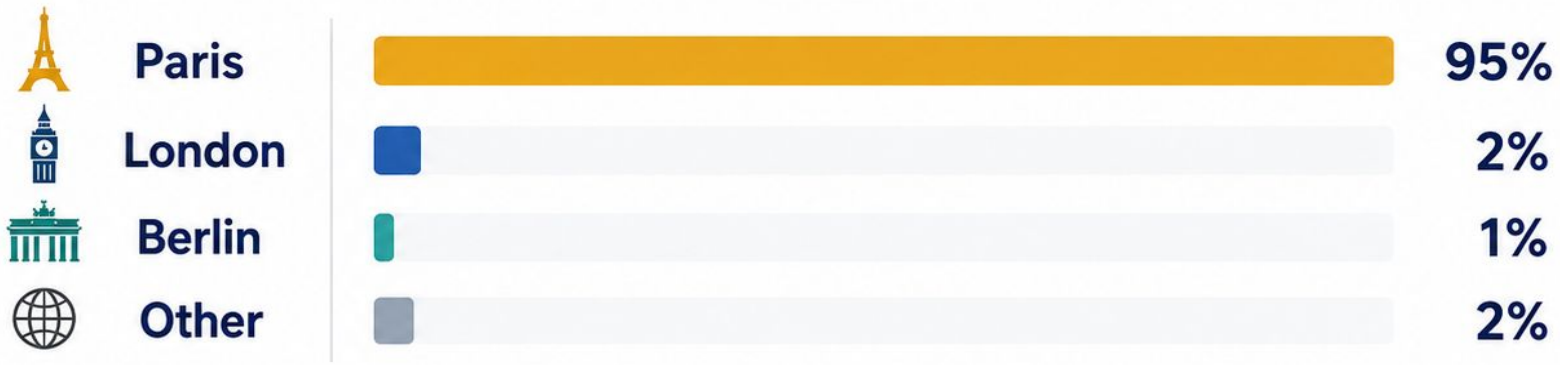
Generative AI is based on:

Learning Probability Distributions

A model learns: *'How likely is each possible output?'*

Example

After: "The capital of France is"



Key Message: The model assigns probabilities to possible outputs and chooses the most likely next token.

What is Probability?

Probability measures the likelihood that an event happens.

- Range:
- $0 \leq$ Probability ≤ 1
- 0 = impossible
- 1 = certain

Examples of Probability

Probability helps us quantify the likelihood of an event happening.

EXAMPLES

Coin

A coin has two possible outcomes: Head or Tail.

HEAD



TAIL



Probability of Head
= 0.5 (50%)

Equal chance of getting Head or Tail.

Dice

A standard dice has 6 possible outcomes: 1, 2, 3, 4, 5, 6.



Probability of rolling 6
= $\frac{1}{6}$ = 0.167 (16.7%)

Only 1 favorable outcome out of 6 possible outcomes.

Weather

Weather forecasts use probability to express uncertainty.

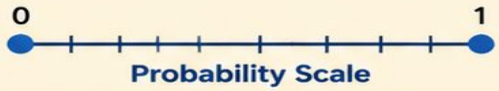


Rain probability
= 70% (0.70)

Means rain is likely, but not certain.

Key Takeaway: Probability values range from 0 to 1.

0 = Impossible to **1 = Certain**



In **Generative AI**, models use **probability** to decide which outputs are more likely and generate new content.

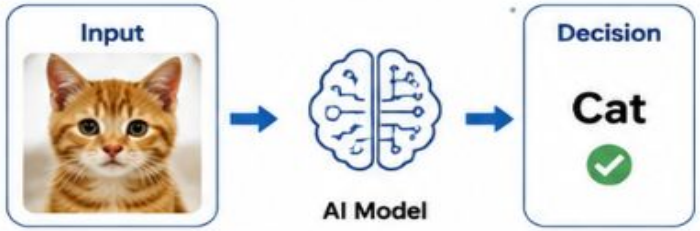
Probability in AI

AI systems use probability to decide or generate outputs.

Traditional AI

Focuses on making a decision or prediction.

Input → Decision



Example

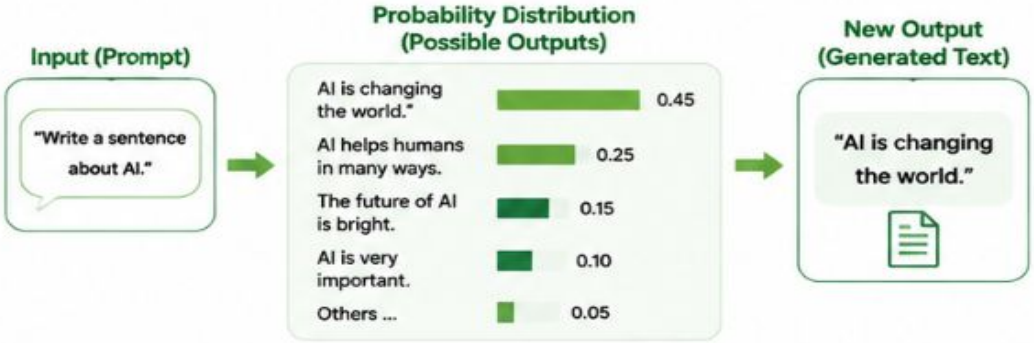


Goal: Find the correct answer / class from given input.

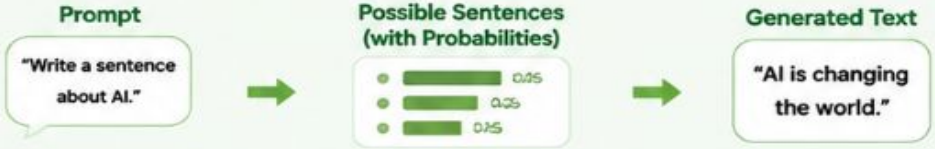
Generative AI

Focuses on generating new content using probabilities.

Input → Probability Distribution → New Output



Example



Goal: Learn the probability of possible outputs and generate new, meaningful content.

Key Difference

Traditional AI predicts the best answer.

Example: Image → Cat

VS

Generative AI generates new content based on probabilities.

Example: Prompt → Possible sentences → Generated text

Activity

Task	Prediction or Generation?
AI detects whether an X-ray shows pneumonia	?
AI writes a patient medical report	?
AI identifies a cat in an image	?
AI creates a new image of a cat	?
AI predicts tomorrow's temperature	?

- X-ray → Prediction
- Medical report → Generation
- Cat detection → Prediction
- New cat image → Generation
- Temperature prediction → Prediction

Activity

- A language model predicts the next word:

- Input:
- "The weather today is“
- "For example:
- sunny → 60%
- rainy → 25%
- cold → 10%
- beautiful → 5%“

- Which word is most likely generated???

Question: A Language Model Predicts the Next Word

Input

"The weather today is"



Language Model

Task

Predict the next word

Possible Outputs and Their Probabilities

Word	Probability
sunny	60%
rainy	25%
cold	10%
beautiful	5%

Key Idea

The model assigns a probability to each possible next word.

The sum of all probabilities = 100%

Explanation:

The language model looks at the input and gives probabilities for all possible next words. The word with the highest probability ("sunny") is the most likely prediction.

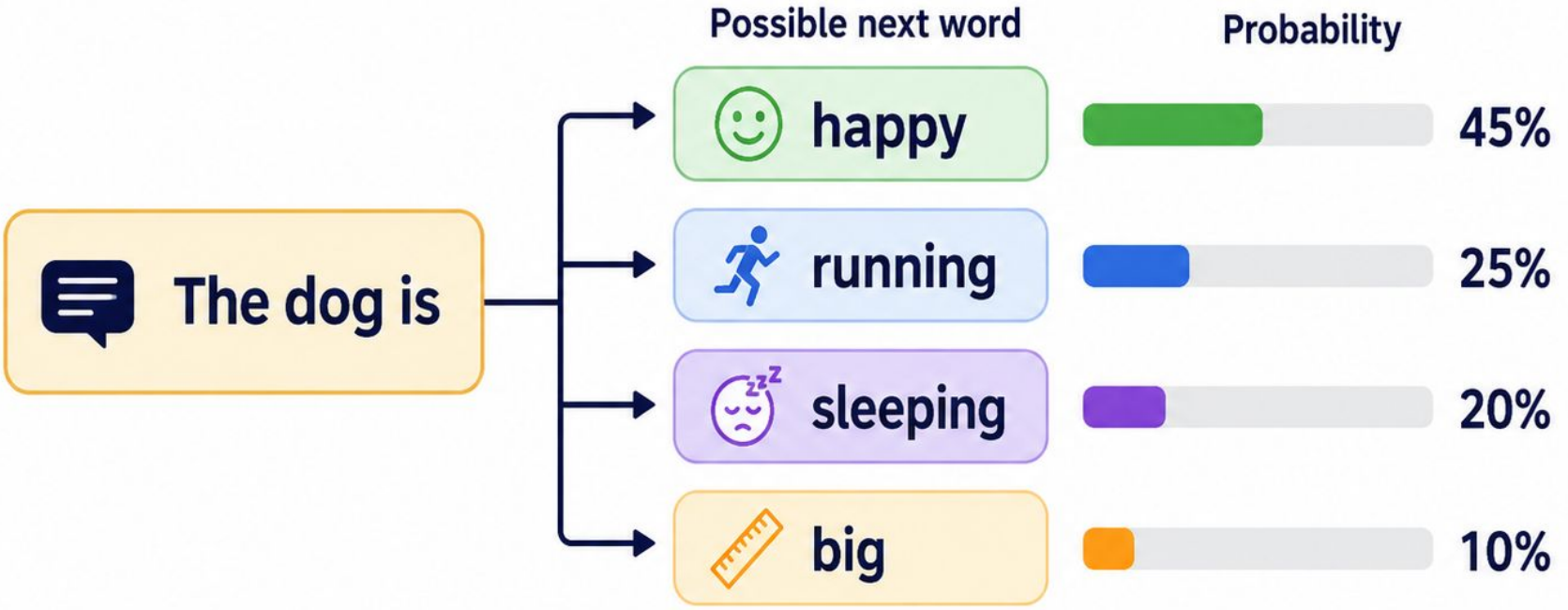
$$\sum 60\% + 25\% + 10\% + 5\% = 100\%$$

Random Variables

- **What is a Random Variable?**
- A variable whose value depends on chance.
- Example:
- Rolling dice:
- Random variable:
- X = Dice result
- Possible values:
- X = {1,2,3,4,5,6}

Random Variables in GenAI

In text generation, the next word is a random variable.



Each possible next word has a probability, and the model selects one based on the learned distribution.

Activity

- Identify the random variable:
- Example:
- Input:
- "I am going to the"
- Possible outputs:
- market
- school
- park
- office

- **What is the random variable in this example?**
- Options:
- A. The sentence
  B. The next word
  C. The AI model
  D. The training data

- **Why?**
- "In this example, the input sentence is fixed:
- **'I am going to the...'**
- The uncertainty is about **what word will come next**."
- "The AI model considers different possible words, such as:
- market
- school
- park
- office
- Each word has a different probability of being selected."
- "Therefore, the **random variable is the next word**, because its value can change depending on chance and probability."
-

- **Why other options are incorrect?**
- ❌ **A. The sentence**
  "The sentence is the input or context provided to the model. It is not uncertain because it is already given."
- ❌ **C. The AI model**
  "The model is the system that calculates probabilities. It does not represent the random outcome."
- ❌ **D. The training data**
  "The training data is used to learn patterns, but it is not the uncertain value being generated."

What is a Probability Distribution?

- A probability distribution shows:
- "How probability is assigned to possible outcomes."
- Example:
- Next word prediction:
- Input:
- "I like"
- Distribution:
- pizza 40%
- music 25%
- coffee 20%
- books 15%

Probability Distribution Example

- Generating an image:
- Prompt:
- "A picture of a dog"
- Possible outputs:
- Golden retriever 50%
- Husky 30%
- Bulldog 20%
- The model samples from this distribution.

Types of Probability Distributions

- Two main types:
- **1. Discrete Distribution**
- Finite possible values
- Example:
- Words
- **2. Continuous Distribution**
- Infinite possible values
- Example:
- Image pixels, audio signals

Types of Probability Distributions

Two main types of probability distributions are used in AI and Generative AI.

1 Discrete Distribution



Finite possible values
The variable can take countable, separate values.

Example: Words

When generating the next word,
the model considers a fixed set of possible words.

Word (Possible Value)	Probability
happy	0.35
running	0.25
sleeping	0.20
big	0.10
other words	0.10



2 Continuous Distribution

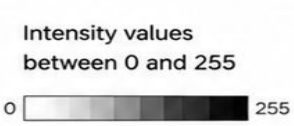


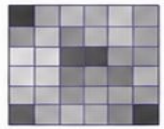
Infinite possible values
The variable can take any value within a range or interval.

Example: Image Pixels, Audio Signals

Pixel intensities or audio amplitudes
can take any real value within a range.

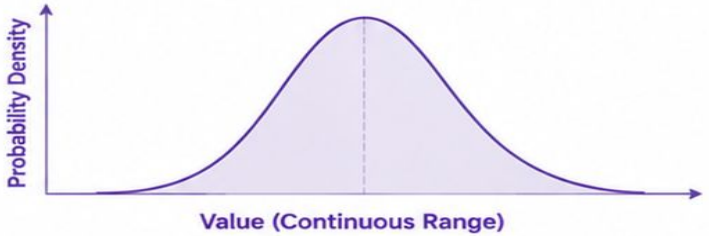
Image pixels: Intensity values between 0 and 255





Audio signals: Amplitude values in a continuous range







Key Takeaway: Discrete distributions deal with countable outcomes (like words), while continuous distributions deal with values in a range (like pixels or audio).

Main Idea of Generative Models

- A generative model tries to learn:
- **P(Data)**
- "The probability distribution of the data."
- Example:
- Images of cats:
- Model learns:
- Shapes
- Colors
- Patterns
- Features

Joint Probability?

- Joint probability means:
- Probability of two events happening together.
- Notation:
- $P(A, B)$
- or
- P(A∩B)

- Events:
- A = It is raining
- B = Person carries umbrella
- Joint probability:
- Probability that:
- "It rains AND person carries umbrella"

Joint Probability in Language Models

1 Sentence

“The cat sits”

2 What the model considers



The language model looks at the entire sentence as a whole and calculates the probability of **all words occurring together**.

3 Joint Probability

P(The, cat, sits)

Probability of the words “The”, “cat”, and “sits” **occurring together in this exact order.**

How it works

Start (beginning) → START

The	×	cat	×	sits	=	P(The, cat, sits)
P(The)		P(cat | The)		P(sits | The, cat)		Joint probability of the full sentence



Joint probability tells us how likely a complete sequence of words is.

Key Takeaway

In language models, joint probability measures the likelihood of all words in a sentence appearing together in the same order.



Language Generation

- A sentence probability:
- $P(w_1, w_2, w_3, \ldots w_n)$
- Example:
- "The dog runs"
- =
- P(The) × P(dog|The) × P(runs|The dog)

Conditional Probability?

- Probability of an event given another event happened.
- Formula:
- $P(A \mid B)$
- Read:
- "Probability of A given B"

- **Conditional Probability in LLMs**
- LLMs generate:
- Next word based on previous words.
- Example:
- "The capital of France is"
- Model calculates:
- $P(\text{word} \mid \text{previouswords})$

Next Token Prediction

1. Input

"The sun rises in the"

The model receives the previous words (context).

2. Model calculates P(next token | previous words)

Possible next tokens (with probabilities)	
morning	80%
sky	10%
east	5%
day	5%

The model assigns a probability to all possible next tokens.

3. Model selects a token

Selected next token:

morning

The token with the highest probability is chosen.

Resulting text after adding the next token:

The sun rises in the **morning**

Maximum Likelihood Estimation

- Goal:
- Find model parameters that make observed data most likely.
- Simple idea:
- Choose the model that best explains the data.

- Two models generate text.
- Model A:
- Probability of training sentences:
- 90%
- Model B:
- Probability:
- 40%
- Which is better?
- Model A.

Sampling

What is Sampling?

Sampling means:

Selecting an output from a probability distribution.

1 Probability Distribution

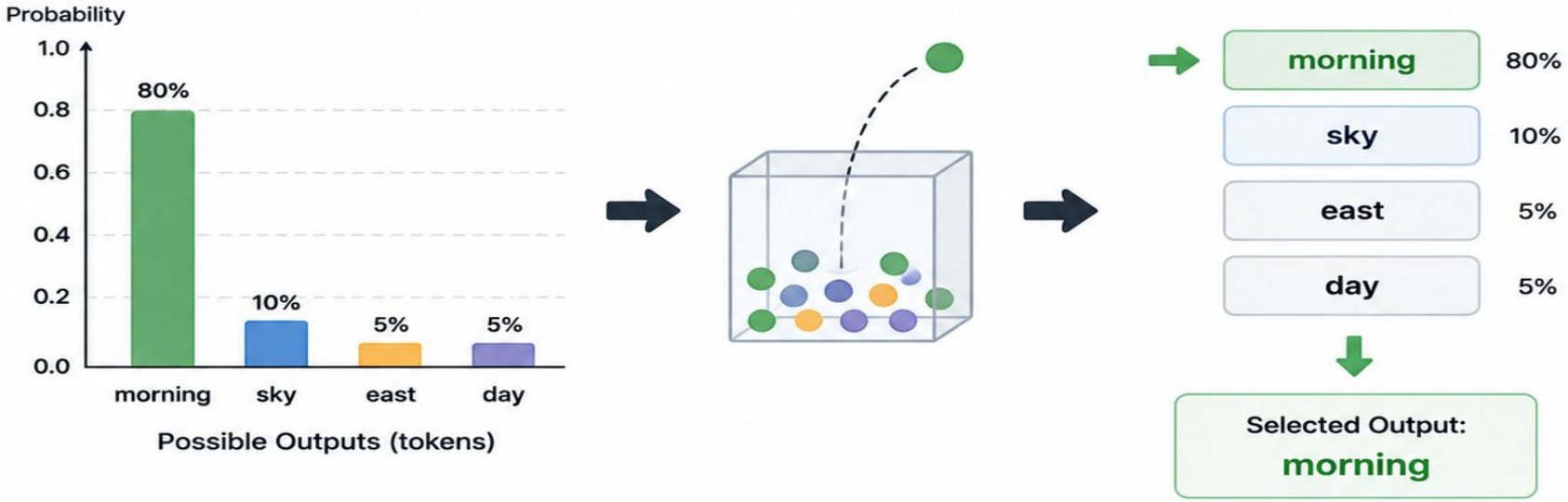
Model assigns probabilities to all possible outputs.

2 Sampling Process

A random draw is made based on these probabilities.

3 Sampled Output

One output is selected as the result.



In Simple Words

Sampling is like picking one ball from a bag, where each ball's chance of being picked is proportional to its probability.

- **Example**
- Distribution:
- Pizza 60%
- Burger 30%
- Pasta 10%
- One sample:
- Pizza
- Another sample:
- Burger

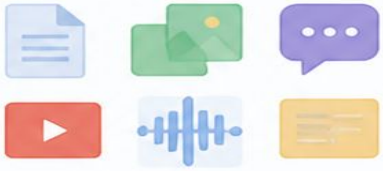
Sampling in Generative AI

- Text generation:
- Probability distribution:
- word1 40%
- word2 30%
- word3 20%
- word4 10%
- Model samples next word.

How Generative Models Work

1. TRAINING

Data



Large amount of training data

Learn probability distribution

Model finds patterns and learns the underlying distribution of the data.

Model



The model now represents the learned probability distribution.

2. GENERATION

Prompt

"Write a short story about a brave knight."

Probability Calculation



Model calculates the probability of possible next tokens (words).

Sampling



One token is randomly selected based on the probabilities.

New Output

"Once upon a time, in a distant land, there lived a brave knight who ..."

The process repeats to generate the complete output.

In Simple Words

The model learns the patterns from data (training). When given a prompt, it predicts what comes next and samples step by step to create new content (generation).

Activity

- Predict next word:
- "The university students are"
- Options:
- A. studying
- B. banana
- C. flying
- Which has highest probability?

Activity

- Generative AI creates images because:
- A. It stores every image
- B. It learns data distribution
- C. It copies images

LEC#5

Autoregressive Generative Models

- **Learning Objectives**
- By the end of the next lecture, students will be able to:
- Understand how autoregressive models generate sequences step-by-step.
- Explain sequence probability and its role in language generation.
- Understand next-token prediction in large language models.
- Explore different decoding strategies used to generate text.

What Are Autoregressive Generative Models?

- Autoregressive models generate new data by predicting the next element based on previously generated elements.
- Key Idea
- Previous tokens → Model → Next token prediction
- Example:
- "The sun rises in the" → morning
- Applications
- Large Language Models (LLMs)
- Text generation
- Machine translation
- Speech generation
- Code generation

Understanding Sequence Generation

Sequence Generation

A sequence is an ordered collection of elements such as words or tokens.

Example:

The → sun → rises → in → the → morning

Autoregressive models generate sequences one step at a time.

Generation Process

1. **Predict first token**
   Model predicts the first token in the sequence.
2. **Use previous tokens as context**
   The generated token(s) become the context for the next prediction.
3. **Predict next token**
   Model predicts the next token based on the current context.
4. **Repeat until completion**
   Repeat the process until an end condition (e.g., end token) is met.

Step-by-Step Visualization

				Next Token Prediction
Step 1	Context (Start)	<START>	Model	The
Step 2	Context	The	Model	sun
Step 3	Context	The sun	Model	rises
Step 4	Context	The sun rises	Model	in
Step 5	Context	The sun rises in	Model	the
Step 6	Context	The sun rises in the	Model	morning
Step 7	Context	The sun rises in the morning	Model	<END>

The model keeps using the previously generated tokens as context to predict the next token, repeating the process until the sequence is complete.

Sequence Probability

Main Concept

The probability of a complete sequence is calculated by multiplying the probability of each token given previous tokens.

The → cat → sleeps → <END>

Formula

$$P(X) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_1, \dots, x_{n-1})$$

$P(X)$ = Probability of the complete sequence X

x_i = i-th token in the sequence

$P(x_i|x_1, \dots, x_{i-1})$ = Probability of current token given all previous tokens

Example

Sentence:

The cat sleeps <END>

Probability:

$$P(\text{The}) \times P(\text{cat}|\text{The}) \times P(\text{sleeps}|\text{The, cat}) \times P(\langle\text{END}\rangle|\text{The, cat, sleeps})$$

Step-by-Step Calculation (Example with Hypothetical Probabilities)

1st token	2nd token given previous	3rd token given previous	4th token given previous	Sequence Probability
The	cat	sleeps	<END>	P(X)
P(The) = 0.30	× P(cat|The) = 0.25	× P(sleeps|The, cat) = 0.60	× P(<END>|The, cat, sleeps) = 0.90	= 0.30 × 0.25 × 0.60 × 0.90 = 0.0405

What does it mean?

The model breaks down the sequence into a chain of conditional probabilities.
By multiplying them, it gets the probability of the entire sequence.
Higher probability → the sequence is more likely according to the model.

Key Takeaway

Autoregressive models evaluate how likely a whole sequence is by considering the context at every step.

Next-Token Prediction

Core Mechanism of LLMs:
LLMs generate text by predicting one token at a time.

Example

Input (Context):

"The capital of France is"

Possible Next-Token Predictions

Token	Probability
Paris	95%
London	2%
Rome	1%
...	...

Selected Token

Paris

The token with the highest probability is selected.

Process Flow

1. **Context**
   Current generated text is used as context.
2. **Probability Calculation**
   Model calculates probability distribution over next possible tokens.
3. **Token Selection**
   Select the token with highest probability (or using a sampling strategy).
4. **Continue Generation**
   Add the selected token to the context and repeat the process.

Step-by-Step Generation Example

Step 1	Step 2	Step 3	Step 4	Step 5
Context: The capital of France is	Context: The capital of France is Paris	Context: The capital of France is Paris the	Context: The capital of France is Paris the capital	Context: The capital of France is Paris the capital city
Predicted Next Token: Paris	Predicted Next Token: the	Predicted Next Token: capital	Predicted Next Token: city	Predicted Next Token: of
(Probability: 95%)	(Probability: 72%)	(Probability: 68%)	(Probability: 74%)	(Probability: 71%)

... and so on

Key Takeaway:
LLMs work by repeatedly predicting the next token based on the current context, forming text one token at a time.

Activity

- Think
- **Prompt:**
- "Artificial intelligence is changing the"

- **What is the most likely next word?**
- Like an AI Mode
- Possible options:
- Token
- Probability
- world
- technology
- future

Possible options:

Token	Probability
world	50%
technology	30%
future	20%

Autoregressive Decoding

- Autoregressive decoding is the process of generating a sequence by repeatedly predicting and adding the next token.
- **Example**
- Prompt:
- "The sky is"
- Step 1:
- "The sky is blue"
- Step 2:
- "The sky is blue and"
- Step 3:
- "The sky is blue and clear"
- **Key Point**
- Each generated token becomes part of the context for predicting the next token.

Activity 2: Build a Sentence

- **Title: "Generate Like an LLM"**
- Start with:
- **Prompt:**
- "The student opened the"
- Students choose the next token:
- Step 1:
- book
- laptop
- door
- Assume selected:
- **book**
- New sequence:
- "The student opened the book"
- Step 2:
- Choose next token:
- and
- because
- yesterday
- Continue for 3–4 steps.
- **Learning Point:**
  Each new token becomes the context for predicting the next token.

ACTIVITY ANSWER: "GENERATE LIKE AN LLM"

PROMPT:

"The student opened the"

STEP 1

Choose next token

OPTIONS:

- book ✓
- laptop
- door

SELECTED TOKEN: book

NEW SEQUENCE:
"The student opened the book"

STEP 2

Choose next token

OPTIONS:

- and ✓
- because
- yesterday

SELECTED TOKEN: and

NEW SEQUENCE:
"The student opened the book and"

STEP 3

Choose next token

OPTIONS:

- started ✓
- closed
- forgot

SELECTED TOKEN: started

NEW SEQUENCE:
"The student opened the book and started"

STEP 4

Choose next token

OPTIONS:

- reading ✓
- running
- sleeping

SELECTED TOKEN: reading

NEW SEQUENCE:
"The student opened the book and started reading."

FINAL GENERATED SENTENCE:

"The student opened the book and started reading."

LEARNING POINT:

Each new token becomes the context for predicting the next token.
This is how autoregressive generation works!

Context (previous tokens) → Predict Next Token → Add to Context & Repeat

Decoding Strategies

- **Why Decoding Strategies?**
- Different strategies control how the model selects the next token.
- **Common Strategies**
- **1. Greedy Decoding**
- Always selects the highest probability token.
- Produces predictable outputs.
- Example:
- 80% token selected every time.

2. Beam Search

- Explores multiple possible sequences.
- Selects the sequence with the highest overall probability.
- Used in:
- Translation
- Text generation

DECODING STRATEGIES

Why Decoding Strategies?
Different strategies control how the model selects the next token.

1. GREEDY DECODING

- Always selects the highest probability token.
- Produces predictable outputs.

Example

Prompt: "The sky is"

At each step, the highest probability token is selected.

80% token selected every time.



Prompt: "The sky is"

Step 1		Step 2		Step 3		Step 4
The sky is	→	The sky is blue	→	The sky is blue and	→	The sky is blue and clear
Next Token	Prob.	Next Token	Prob.	Next Token	Prob.	End of sequence
blue	80%	and	70%	clear	75%	
very	10%	but	15%	bright	10%	
really	5%	with	10%	calm	10%	
...	...	...	...	...	...	

Always pick the highest probability token at each step

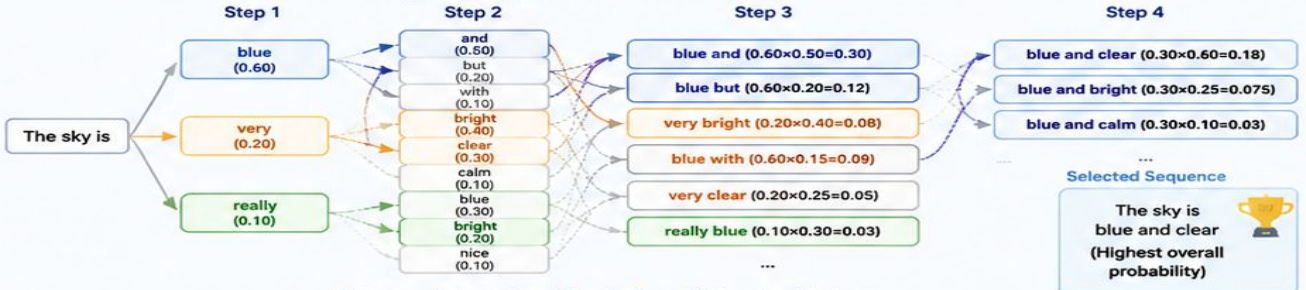
2. BEAM SEARCH

- Explores multiple possible sequences.
- Selects the sequence with the highest overall probability.

Used in

- Translation
- Text Generation

Prompt: "The sky is" | **Beam width = 3**



Beam search keeps top-k (here k=3) candidates at each step

3. SAMPLING-BASED DECODING

- Selects tokens based on probability distribution.
- Produces diverse outputs.

Example

Prompt: "Good"

Token	Probability
morning	80%
sky	10%
east	5%
others	5%

How it works (one possible run)



Different runs can produce different but plausible outputs

Comparison of Decoding Strategies

Strategy	How it works	Pros	Cons	Best used for
Greedy Decoding	Always picks the highest probability token.	Fast, simple, predictable	May miss better long-term sequences	When speed and consistency are important
Beam Search	Explores multiple sequences and picks the best overall.	Better quality, more accurate	Slower, more computational cost	Translation, summarization, high-quality generation
Sampling-Based Decoding	Samples tokens based on probability distribution.	More diverse, creative outputs	May produce less predictable results	Creative writing, chatbots, exploratory generation

Greedy vs Sampling

- Prompt:
- "The robot walked into the"
- **Greedy Decoding Output:**
- "The robot walked into the room."
- **Sampling Output:**
- "The robot walked into the mysterious forest."

Complete Workflow of Autoregressive Generation

From Prompt to Output

1. Prompt

User provides the initial input or instruction.

Example:
"The sky is"

2. Context Representation

The model converts the prompt and previous tokens into a context representation.

Tokens ...
Context Vector →

3. Calculate Next-Token Probabilities

The model computes a probability distribution over the entire vocabulary.

blue	0.45
clear	0.20
bright	0.10
...	...
other	0.25

4. Apply Decoding Strategy

A decoding strategy (e.g., greedy, beam search, sampling) is applied to the probabilities.

Strategies

- Greedy Decoding
- Beam Search
- Sampling-Based Decoding

5. Select Next Token

A token is selected based on the chosen decoding strategy.

Example:
Selected token → "blue"

6. Repeat Process

The selected token is added to the context, and the process repeats for the next token.

The | sky | is | blue | ?

7. Generated Text

The process continues until a stopping criterion is met (e.g., end token or max length).

Example Output:
"The sky is blue and clear."

Design an AI Text Generator

- **Chatbot**
- What is the input?
- What tokens will the model predict?
- Which decoding strategy would you choose?

Lec#6

Classical Neural Generative Models: Autoencoders, VAEs, and GANs

- **Learning Objectives**
- By the end of this lecture, students will be able to:
- Understand the basic principles of neural generative models.
- Explain the concept of latent space and representation learning.
- Describe the working principles of Autoencoders, VAEs, and GANs.
- Compare classical generative models with modern approaches such as LLMs.
- Identify the strengths and limitations of classical generative models.

- **What Are Neural Generative Models?**
- **Definition**
- Neural generative models are deep learning models that learn patterns from existing data and generate new samples with similar characteristics.
- **Main Idea**
- **Data → Learn hidden patterns → Generate new data**

What Are Neural Generative Models?

DEFINITION

Neural generative models are deep learning models that learn patterns from existing data and generate new samples with similar characteristics.

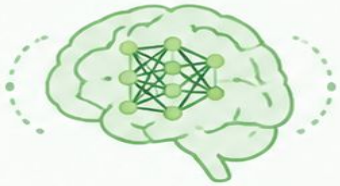
MAIN IDEA

DATA



Existing data

LEARN HIDDEN PATTERNS



Model learns the underlying patterns and structure

GENERATE NEW DATA

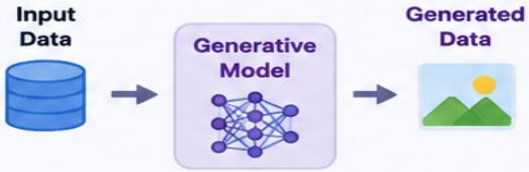


New samples with similar characteristics

EXAMPLES / APPLICATIONS

- Generate images
- Create synthetic data
- Learn meaningful representations

HOW IT WORKS



The model learns the data distribution and generates new, similar data.

EXAMPLES OF MODELS

- Autoencoders (AE)
- Variational Autoencoders (VAE)
- Generative Adversarial Networks (GANs)

KEY TAKEAWAY:
Neural generative models learn from data and can create new, realistic samples.
They are the foundation of many modern AI applications.

Concept of Latent Space

What is Latent Space?

Latent space is a compressed representation where a model stores important features of data.

Example: Image of a face

Original Image



Latent Representation

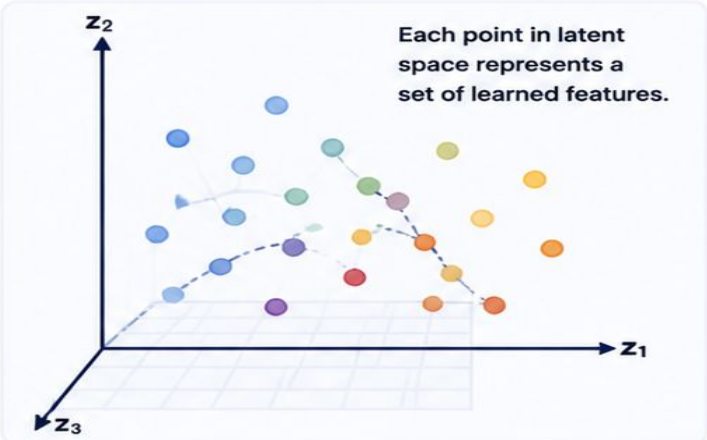


Features



Generated Face





Key Idea

Models learn hidden structures instead of memorizing data.



Activity : Discover Latent Features

- **Think Like a Neural Network**
- Input:
- Images of different animals:
-  Cat
   Dog
   Tiger
- **What hidden features should the model learn?**